

无线通信安全第二讲作业

1. 在 AES 分组密码中，若密文出现 1 比特差错，分析不同工作模式（ECB、CBC、CFB、OFB、CTR）下的差错传播特性（即影响的比特数），并说明无线信道加密中应如何合适的工作模式及其原因。

(1) 分析不同工作模式的差错传播特性

①ECB 模式中，各分组之间互不影响：

$$P_i = D_k(C_i)$$

所以如果某一密文分组有 1 比特出错，那么只会影响这一个分组对应的明文。由于分组解密后会产生扩散，所以一般会使该明文分组全部出错，但不会影响后面的分组。

②CBC 模式中：

$$P_i = D_k(C_i) \oplus C_{i-1}$$

如果某一密文分组有 1 比特出错，那么会使当前分组解密后得到的明文整个分组出错。同时：

$$P_{i+1} = D_k(C_{i+1}) \oplus C_i$$

因为下一组明文要和这一组密文异或，所以还会使下一明文分组中对应的 1 比特出错，再后面的分组不受影响。

③CFB 模式中，密文会反馈到后面的加密过程中：

$$P_i = C_i \oplus E_k(C_{i-1})$$

因此如果某一密文有 1 比特出错，首先会使当前明文中对应的 1 比特出错，然后这个错误还会继续影响后面若干比特或若干分组：

$$P_{i+1} = C_{i+1} \oplus E_k(C_i)$$

等错误被移出反馈寄存器后才恢复正常。所以 CFB 模式会有一段时间的差错传播。

④OFB 模式中：

$$P_i = C_i \oplus K_i$$

反馈的是输出，不是密文，所以密文中的差错不会影响后面的密钥流生成。如果密文有 1 比特出错，那么解密后也只是使对应的 1 比特出错，不会影响其他比特，也不会影响后面的分组。

⑤CTR 模式中：

$$P_i = C_i \oplus E_k(\text{Counter}_i)$$

每个分组都是把计数器加密后得到密钥流，再和密文异或，所以各分组之间也是相互独立的。如果密文中 1 比特出错，那么解密后也只是使对应的 1 比

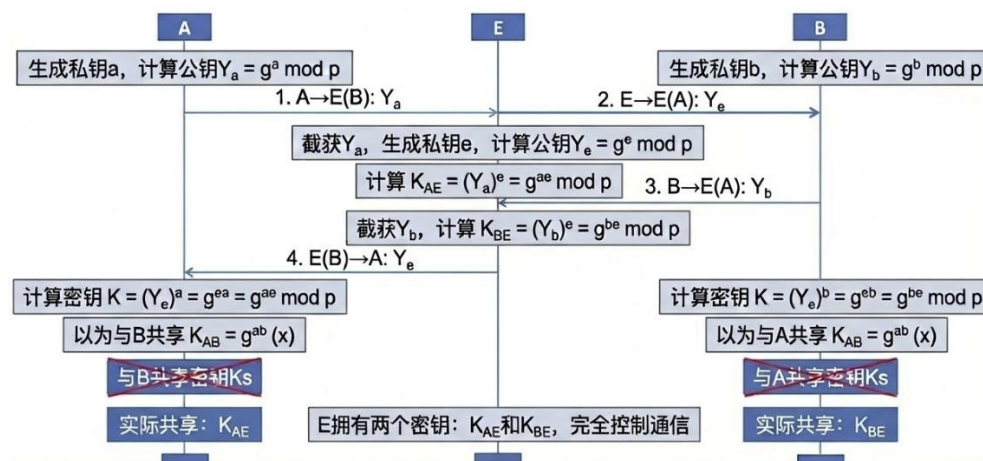
特出错，不会扩散到其他位置。

(2) 在无线信道中，由于信道容易受到干扰，常常会出现比特差错，所以应尽量选择差错不扩散的工作模式。这样即使传输中某一位出错，也不会影响太多数据。因此，无线信道加密一般应选择 OFB 模式或 CTR 模式，因为这两种模式中，明文满足：

$$P_i = C_i \oplus \text{keystream}$$

密文 1 比特出错时，只会导致明文对应 1 比特出错，不会引起更多错误传播，更适合误码率较高的无线通信环境。

2. 请按照课件第 75 页 PPT 的示例形式，给出针对 DH 密钥交换协议的一种攻击。



设通信双方为 A 和 B，攻击者为 E。

(1) 正常 DH 协议过程：

$A \rightarrow B: g^a \bmod p$ 、 $B \rightarrow A: g^b \bmod p$

A 计算： $K = (g^b)^a = g^{ab} \bmod p$ 、B 计算： $K = (g^a)^b = g^{ab} \bmod p$

(2) 中间人攻击过程：

$A \rightarrow E(B): g^a$ ：A 发送自己的公钥值 g^a ，但被攻击者 E 截获。

$E(A) \rightarrow B: g^e$ ：E 生成自己的私钥 e ，把 g^e 发送给 B，冒充 A。

$B \rightarrow E(A): g^b$ ：B 将自己的公钥值 g^b 发送给 A，但被 E 截获。

$E(B) \rightarrow A: g^e$ ：E 再把自己的 g^e 发送给 A，冒充 B。

(3) 此时各方计算的密钥：

A 计算： $K_{AE} = (g^e)^a = g^{ae}$ 、B 计算： $K_{BE} = (g^e)^b = g^{be}$

E 可以计算：与 A 的密钥： g^{ae} 、与 B 的密钥： g^{be}

(4) 攻击结果：A 以为和 B 建立了密钥，实际上是和 E 建立了密钥；B 也以为和 A 建立了密钥，实际上是和 E 建立了密钥；攻击者 E 可以：解密 A 发送给 B 的数据；再加密后转发给 B；反之亦然。因此，通信完全被 E 控制，而 A 和 B 都无法察觉。

3. 请分析基于公钥的 NS 协议（PPT 第 78 页）的安全性，指出其存在的问题，并给出改进协议或建议。

(1) 基于公钥的 NS 协议的基本过程是：

$$A \rightarrow B: E_{K_{Ub}}(ID_A, N1)、B \rightarrow A: E_{K_{Ua}}(N1, N2)、A \rightarrow B: E_{K_{Ub}}(N2)$$

这个协议原来的目的是让 A 和 B 通过随机数 $N1$ 、 $N2$ 相互认证，再进一步建立会话密钥。但是，这个协议并不是完全安全的，它存在一个经典问题，就是容易受到中间人攻击（Lowe 攻击）。

假设攻击者是 I ，攻击过程可以简单表示为：

$A \rightarrow I(B): E_{K_{Ui}}(ID_A, N1)$ ：A 以为自己在和 I 通信，或者攻击者诱使 A 发起一次会话。

$I(A) \rightarrow B: E_{K_{Ub}}(ID_A, N1)$ ：攻击者把 A 发来的内容转发给 B，冒充 A。

$B \rightarrow A: E_{K_{Ua}}(N1, N2)$ ：B 认为自己正在和 A 通信。

$A \rightarrow I(B): E_{K_{Ui}}(N2)$ ：A 验证 $N1$ 正确后，把 $N2$ 返回给攻击者。

$I(A) \rightarrow B: E_{K_{Ub}}(N2)$ ：攻击者再把这个结果转发给 B。

这样一来，B 就会误认为和自己通信的人就是 A，于是认证成功。实际上，攻击者夹在中间完成了欺骗。所以说，这个协议的问题是：协议消息中对通信对方身份的绑定不够，导致 A 无法确认第二步消息到底是不是 B 发来的。更具体地说，第二步消息只有 $N1, N2$ ，没有把 B 的身份信息带进去，因此 A 虽然能验证 $N1$ 是自己发出的，但不能确定这条消息一定来自 B。攻击者正是利用了这一点。**即不能抵抗中间人攻击。**第二条消息没有包含响应方身份，身份绑定不充分。协议只验证了随机数的新鲜性，但没有把“是谁发来的”说明清楚。因此会出现“B 以为自己认证了 A，实际上是攻击者在转发消息”的情况。

(2) 改进方法：第二步中加入 B 的身份，改为

$$A \rightarrow B: E_{K_{Ub}}(A, N_A)、B \rightarrow A: E_{K_{Ua}}(N_A, N_B, B)、A \rightarrow B: E_{K_{Ub}}(N_B)$$

这样 A 在收到第二条消息后，不但能检查 N_A 是否正确，还能检查消息中是否带有 B 的身份。如果攻击者转发消息，由于身份不对应，A 就能发现异常，从而防止中间人攻击。这就是著名的 Lowe 改进方案。

(3) 进一步建议

- ①在协议消息中加入双方身份信息，避免身份混淆。
- ②加入时间戳或会话标识，防止旧消息重放。
- ③结合数字签名和证书机制，确认公钥确实属于对应用户，避免公钥被替换。
- ④在实际系统中，最好不要直接使用原始 NS 协议，而应使用已经过广泛验证的认证密钥交换协议，例如 TLS、IKE、Kerberos 改进方案等。